Johdanto.

Tietokannat tietojärjestelmässä. SQL-kertaus

Oppimistavoitteet

1. Ymmärretään tietokannan rooli tietojärjestelmässä
2. Kerrataan ja syvennetään SQL-kielen hallintaa.

Lähtökohdat ja määritelmät

- Tietokanta (*database*) tarkoitetaan järjestettyä kokoelmaa toisiinsa liittyvää tietoa.
  - Esim. verkkokaupan asiakas-, tuote- ja tilaustiedot muodostavat tietokannan
- Tietokannan hallintajärjestelmä (*database management system, DBMS*) eli tietokantaohjelmisto viittaa ohjelmistoon, jolla tietokantaa käytetään ja hallinnoidaan.
  - Esim. MariaDB on tietokannan hallintajärjestelmä.
- Tietokantajärjestelmä (*database system*) koostuu tietokannasta ja tietokannan hallintajärjestelmästä.
- Tietojärjestelmä (*information system*) tarkoittaa ohjelmistojen, tietokantajärjestelmän, prosessien ja (yleensä) ihmisten muodostamaa kokonaisuutta, joka palvelee tiettyä liiketoimintatavoitetta.

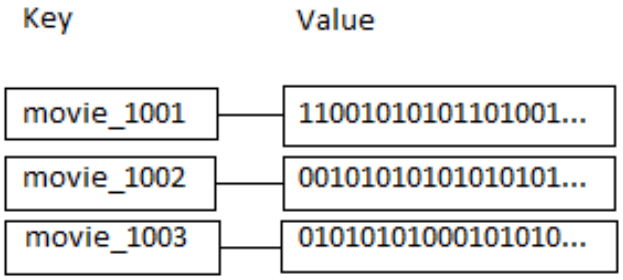
Kehitystyön lähtökohtia

- Tietojärjestelmiä rakennetaan harvoin puhtaalta pöydältä, jolloin mukana on historian painolastia.
- Tietokannan elinkaari on usein pitempi kuin sitä käyttävien ohjelmistojen.
- Sama tietokanta palvelee usein suurta joukkoa ohjelmia.
- Tästä seuraa, että isoissa tietojärjestelmissä kehitystyö tehdään tyypillisesti database first -periaatteella.
  - Vrt. software first –lähestymistapa, jossa tietokanta on alisteinen sitä käyttävän ohjelmiston ratkaisuille.

Tietokantamalli

- Tietokantamalli määrää tavan, jolla kohdealueen tieto kuvataan.
- Tietokantamalleja
  - relaatiotietokantamalli
    - yleisin ja laajimmin käytetty tietokantamalli
  - oliotietokantamalli
    - kiinnostava oliopohjaisten ohjelmointikielten ansiosta.
    - ei levinnyt laajaan käyttöön, standardoinnin puute.
    - Käytännössä tehdään yleensä olio-relaatiomuunnos (ORM, *object-relational mapping*, jolloin ohjelmointikielen oliot tallennetaan relaatiotietokantaan.
  - NoSQL-tietomallit
    - Data tallennetaan esim. avain-arvopareina.
    - Vähän oletuksia datan rakenteesta.
    - Soveltuvat erittäin suurille tietoaineistoille, joihin kohdistuu vähän päivityksiä (ns. *big data*)

NoSQL-tietomallit



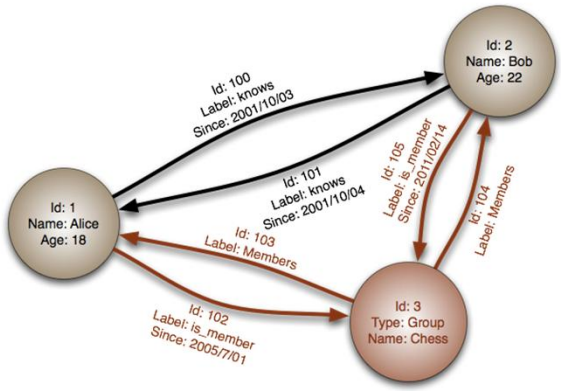
Avain-arvo-tietokannat

```
{"students":[
     {"FirstName": "Mary", "LastName: "Doe"},
     {"FirstName": "John", "LastName: "Doe"},
     {"FirstName": "Marlene", "LastName: "Smith"}
]}
```

Dokumenttitietokannat



Sarakeperhetietokannat

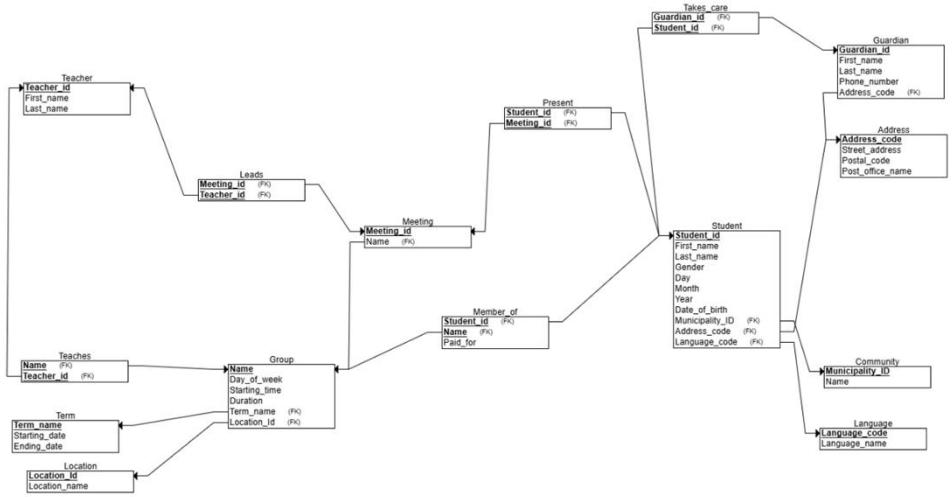


Verkkotietokannat

Kuvien lähteet: Ahzf (Transferred by Obersachse) , via Wikimedia Commons, public domain ja Sae1962, via Wikimedia Commons, under Creative Commons Attribution 3.0 License.

Relaatiotietokantamalli

- Data esitetään taulukkomuodossa.
- Käsitteitä: taulukot, kentät, perus- ja viiteavaimet
- Perustuu kohdealueen käsitteelliseen mallinnukseen.
  - Riippumaton tallennusratkaisusta ja tietovälineestä.



Looginen ja fyysinen tietokantasuunnittelu

- Looginen tietokantasuunnittelussa määritetään tietokannan rakenne datan sisäistä rakennetta heijastavaksi.
  - Relaatiotietokannassa tähän kuuluu taulukoiden, kenttien, niiden tietotyyppien sekä perus- ja viiteavainten määritys.
  - Näin määritetään skeema eli relaatiotietokantakaavio.
- Fyysinen tietokantasuunnittelu tähtää tietokannan tekniseen toteuttamiseen siten, että se on tehokas, skaalautuva ja turvallinen.

SQL:n rooli

- Aiemmissa opinnoissa on opittu perusteet SQL-kyselykielestä.
- SQL:n käyttötarkoitukset:
  1. Datan kyselykieli
     - SELECT
  2. Datan manipulointikieli
     - INSERT, UPDATE, DELETE
  3. Datan rakenteen kuvauskieli
     - CREATE TABLE/INDEX/VIEW
  4. Tietokantapalvelimen ylläpitokieli
     - GRANT/REVOKE, järjestelmäasetusten ylläpito.

SQL-kyselykielenä

- Kerrataan/opitaan SQL:n hakukyselyiden (SELECT) keskeiset seikat ja rakennuspalikat:
  1. Yhden taulun hakukysely
  2. Valintaehdot
  3. Sisä- ja ulkoliitokset
  4. Ryhmittely, koostefunktiot, ryhmien poiminta
  5. Sisäkyselyt
  6. UNION, EXCEPT, INTERSECT
- Näiden avulla on toteutettavissa käytännössä kaikki relaatiotietokantaan kohdistuvat hakutarpeet.

Esimerkkitietokanta



- Tunnista skeemasta
  - perusavaimet
  - viiteavaimet
- Tietokannassa vallitsee viite-eheys. Mitä se tarkoittaa?

Yhden taulun hakukysely ja lajittelu

- SELECT-sanan perässä luetellaan tulosjoukkoon tulevat kenttänimet.
- FROM määrää joukon, josta tulos tuotetaan.
  - Tässä yksi taulu.
- ORDER BY lajittelee
  - Lajittelukriteerejä sovelletaan vasemmalta oikealle.
  - Nouseva tai laskeva järjestys määrätään ASC- tai DESC-määreellä.

```
1 SELECT Etunimi, Sukunimi
2 FROM TYONTEKIJA
3 ORDER BY Sukunimi, Etunimi
```

Etunimi	Sukunimi
Tuula	Aalto
Annika	Aaltonen
Kirsti	Aaltonen
Leevi	Aaltonen
Liisi	Aaltonen
Saara	Aaltonen
Salli	Aaltonen
Sulo	Aaltonen
Suvi	Aaltonen
Valio	Aamuvuori

Valintaehdot

- Valintaehto rajaa tulosjoukkoon kuuluvia tietueita.
- Jokaisen tietueen kohdalta katsotaan erikseen, toteutuuko valintaehto.
- Käytettävissä sulkeet ja konjunktiot (AND, OR, NOT, BETWEEN, ...)

```
1 SELECT Rekisteritunnus, Merkki
2 FROM AUTO
3 WHERE Merkki LIKE 'M%';
```

Rekisteritunnus	Merkki
GDD-852	Mercedes-Benz
NBV-582	Mercedes-Benz

Sisäliitokset

- Sisäliitos INNER JOIN liittää eri taulujen tietueet yhteen yhteisten arvojen perusteella.

```
1 SELECT Etunimi, Sukunimi, OSASTO.Nimi
2 FROM TYONTEKIJA INNER JOIN OSASTO
3 ON TYONTEKIJA.Osasto = OSASTO.Numero
```

Etunimi	Sukunimi	Nimi
Liisi	Virtanen	Markkinointi
Oiva	Ahola	Markkinointi
Taina	Palmu	Markkinointi
Vappu	Lepistö	Markkinointi
Toimi	Priha	Markkinointi
Edvard	Öijälä	Markkinointi
Päivikki	Saoud	Markkinointi

Ulkoliitokset

- Ulkoliitos LEFT/RIGHT JOIN hakee kaikki tietueet vasemman/oikean puolen taulusta riippumatta siitä, onko niillä vastinparia.

```
1 SELECT Etunimi, Sukunimi, POSTITUS.Postinumero, POSTITUS.Postitoimipaikka
2 FROM TYONTEKIJA RIGHT JOIN POSTITUS
3 ON TYONTEKIJA.Postino = POSTITUS.Postinumero
```

Etunimi	Sukunimi	Postinumero	Postitoimipaikka
Lassi	Raunio	02680	Espoo
Ainikki	Pietarinen	02680	Espoo
Aune	Väliahdet	02690	Espoo
Iida	Kleemann	02690	Espoo
Urmas	Niskanen	02690	Espoo
Niina	Rissanen	02690	Espoo
Iida	Toivonen	02690	Espoo
Miska	Joukainen	02690	Espoo
Soila	Skrifvas	02690	Espoo
Julia	Suni	02690	Espoo
Taisto	Saulio	02690	Espoo
Mervi	Summanen	02690	Espoo
Aija	Kiiski	02690	Espoo
Eila	RAITIO	02690	Espoo
Miia	Lassila	02690	Espoo
Eine	Hiltunen-Hyrkäs	02690	Espoo
Hannu	Seppänen	02690	Espoo
(NULL)	(NULL)	02700	Espoo
(NULL)	(NULL)	02710	Espoo
(NULL)	(NULL)	02720	Espoo

Ryhmittely, koostefunktiot, ryhmien poiminta

- GROUP BY ryhmittelee tulosjoukon tietueet sen perässä lueteltujen kenttien arvon mukaan.
- Tällöin SELECT-sanan perässä saa esiintyä:
  - Ryhmittelykenttien nimiä (samoja, jotka GROUP BY -määreen perässä)
  - Koostefunktioita (MIN, MAX, SUM, AVG, COUNT, ...)

```
1 SELECT Merkki, COUNT(*)
2 FROM AUTO
3 GROUP BY Merkki
```

Merkki	COUNT(*)
Fiat	31
Honda	59
Mercedes-Benz	2
Opel	25

Sisäkyselyt

- Sisäkyselyt sallivat yhden kyselyn tulosjoukon käyttämisen toisen kyselyn lähtökohtana.
- Mihin kysymykseen oheinen kysely vastaa?
- Miten tulos lasketaan?

```
1 SELECT Etunimi, Sukunimi
2 FROM TYONTEKIJA INNER JOIN AUTO
3 ON TYONTEKIJA.Numero = AUTO.Haltija
4 WHERE AUTO.Merkki = (
5    SELECT Merkki
6    FROM TYONTEKIJA INNER JOIN AUTO
7    ON TYONTEKIJA.Numero = AUTO.Haltija
8    WHERE Etunimi = 'Vihtori' AND Sukunimi = 'Liukku-Kananen'
9 )
10 ORDER BY Sukunimi, Etunimi
```

Etunimi	Sukunimi
Mari	Alakurtti
Simo	Alanen
Tuulia	Aronen
Lempi	Hassel
Aarne	Havas
Talvikki	Heiskanen
Hannu	Hiltunen
Elena	Hirvonen
Taisto	Huovinen
Ilmo	Höyhtyä

UNION, EXCEPT, INTERSECT

- UNION liittää kaksi vastaavat kentät sisältävää tulosjoukkoa toisiinsa.
- EXCEPT vähentää ensimmäisestä tulosjoukosta ne, jotka esiintyvät toisessa.
  - MariaDB tukee versiosta 10.3.0 alkaen.
  - Joissain tiedonhallintaohjelmistoissa MINUS
- INTERSECT tuottaa leikkauksen eli tietueet, jotka esiintyvät molemmissa tulosjoukoissa.
  - MariaDB tukee versiosta 10.3.0 alkaen.
- ALL-lisämääre (esim. UNION ALL) säilyttää eri osapuolista tulevat duplikaatit

```
1 SELECT Etunimi
2 FROM TYONTEKIJA
3 WHERE Palkka =2138
4 UNION
5 SELECT Etunimi
6 FROM TYONTEKIJA INNER JOIN OSASTO
7 WHERE Nimi="Markkinointi"
```

Etunimi
Tilda
Pekko
Liisi
Mimmi
Veikko
Oiva
Armas
Henrik

Deskriptiivisyys

- SQL on deskriptiivinen kyselykieli.
  - Kyselyn voi periaatteessa ilmaista millä tahansa tavalla, joka tuottaa oikean tulosjoukon.
  - Kyselyn optimoija (tiedonhallintaohjelmiston osa) optimoi kyselyn automaattisesti siten, että sen suoritus on mahdollisimman nopeaa.
  - Hakukyselyiden suoritusnopeutta voidaan tietyissä tilanteissa parantaa indeksoimalla. Tähän palataan myöhemmin.

Tietokannan luonti

- Tietokannan luonti koostuu:
  - Tietokannan perustamisesta (CREATE DATABASE)
  - Taulukoiden luonnista (CREATE TABLE)
  - Indeksien luonnista (CREATE INDEX)
  - Näkymien luonnista (CREATE VIEW)
- Lisäksi tietokantaan annetaan tarvittavat käyttöoikeudet.
  - Lähtökohtana pienimmät tarvittavat oikeudet
    - Sovellukselle annetaan yleensä CRUD-oikeudet (Create, Read, Update, Delete eli SQL-kielellä INSERT, SELECT, UPDATE, DELETE) mutta ei datan rakenteen muokkausoikeuksia.
    - Joissain tilanteissa riittää pelkät lukuoikeudet.

Tietokannan luonti

- Tietokannan luontilauseet kerätään tavalla tai toisella SQL-komennoista koostuvaksi skriptiksi.
- Skripti voidaan tuottaa eri tavoin:
  - Voidaan kirjoittaa suoraan esim. tekstieditorilla.
  - Taulukot voidaan tehdä visuaalisesti (esim. HeidiSQL) ja vedostaa lopputuloksesta tietokantaskripti.
    - Virhealttiimpaa, ei välttämättä helpompaa.
    - HeidiSQL:ssä Tools / Export Database as SQL.
    - Komentoikkunassa mysqldump-komennolla.
- Skripti voidaan suorittaa:
  - HeidiSQL:ssä File/ Load SQL file
  - Komentoikkunassa uudelleenohjaamalla komentotulkin syötevirta siten, että se otetaan tiedostosta: **`mysql tietokanta -u tunnus -p < skripti.sql`**

Tietokannan luonti

```
DROP DATABASE IF EXISTS Firma;
CREATE DATABASE Firma;
USE Firma;
```

- Tietokanta luodaan CREATE DATABASE -lauseella.
- Se poistetaan DROP DATABASE -lauseella.
- Yllä olevat lauseet poistavat mahdollisen vanhan tietokannan, luovat uuden kannan samalla nimellä sekä vaihtavat aktiiviseksi tietokannaksi juuri luodun tietokannan.
  - USE saa aikaan, että samasta istunnosta annettavat myöhemmät SQL-lauseet kohdistuvat Firma-kantaan.

Taulujen luonti

```
CREATE TABLE TYONTEKIJA (
    Numero INT NOT NULL,
    Sukunimi VARCHAR(40) NOT NULL,
    Etunimi VARCHAR(40) NOT NULL,
    Osasto INT,
    Lahios VARCHAR(40),
    Postino VARCHAR(5) NOT NULL,
    Puhelin VARCHAR(16),
    Palkka INT,
    PRIMARY KEY (Numero),
    FOREIGN KEY (Postino) REFERENCES POSTITUS(Postinumero),
    FOREIGN KEY (Osasto) REFERENCES OSASTO(Numero)
);
```

- Jokaisesta tietokantataulusta tehdään CREATE TABLE -lause, joka luo uuden tietokantataulun.
  - Samalla määritellään kentät tietotyyppeineen sekä perusavain ja mahdolliset viiteavaimet.
- Jos perus- ja viiteavaimet koostuvat useasta kentästä, erotetaan kentät pilkuilla toisistaan.
- Viiteavaimia voi olla monta
  - FOREIGN KEY -lauseet erotetaan tällöin toisistaan pilkulla.
- MariaDB tukee viite-eheyttä vain, jos käytössä on sitä tukeva tietokantamoottori (esim. InnoDB).
  - Voi olla palvelimen oletusarvona.
  - Voidaan määrittää taulukohtaisesti määreellä **`ENGINE=InnoDB`**, joka kirjoitetaan aivan CREATE TABLE -lauseen loppuun, päättävän kaarisulkeen jälkeen.

Datan lisäys tietokantaan

```
CREATE TABLE OSASTO (
        Numero INT NOT NULL,
        Nimi VARCHAR(40) NOT NULL,
        Budjetti INT,
        PRIMARY KEY (Numero)
);

INSERT INTO OSASTO(Numero,Nimi) VALUES (1, "Markkinointi");
INSERT INTO OSASTO VALUES (2, "Laskutus", 90000);
```

- Tietokannan luonnin yhteydessä tietokantaan halutaan joskus lisätä tietueita eli pohjadataa.
- Yllä on OSASTO-taulun lisäyslause sekä kaksi erilaista tietueen lisäyslausetta. Kumpikin kysely lisää uuden tietueen OSASTO-tauluun.
  - Ensimmäisessä INSERT-lauseessa täsmennetään kentät, jotka saavat arvot.
  - Toisessa INSERT-lauseessa annetaan arvo kaikille kentille.

Datan lisäys tietokantaan (toinen tapa)

```
LOAD DATA LOCAL INFILE 'tyontekijat.txt' into table TYONTEKIJA fields terminated
by ';' (Numero,Sukunimi,Etunimi,Osasto,Lahios,Postino,Puhelin,Palkka);
```

tyontekijat.txt

```
1;Mynttinen;Pekko;2;Puistoraitti 1;00520;040-584512;2133
2;Virtanen;Liisi;1;Ruusuraitti 18;00960;050-296767;4461
3;Rojola;Mimmi;2;Lehmustie 10;01150;050-085274;5008
4;Mikkonen;Veikko;2;Käpypuisto 19;00690;040-556145;2323
5;Ahola;Oiva;1;Mäntypolku 2;01020;050-160969;3736
6;Mäki-Tuuri;Armas;4;Puistopuisto 22;01070;050-149281;2368
7;Joeli;Henrik;4;Jalavapolku 5;02670;040-067335;4259
8;Palmu;Taina;1;Kukkaraitti 3;02630;042-006705;2188
9;Koivistoinen;Ulpu;5;Puistokatu 9;00380;042-637850;2618
10;Huovila;Reima;4;Katajatie 16;01550;041-918868;5246
```

- MariaDB-taulukoihin tulevat tiedot voidaan lukea myös tekstitiedostoista.
- Tuonti INSERT-lausein on kuitenkin standardinmukainen tapa.

Käyttäjätilien määritys

```
CREATE USER 'dbuser'@'localhost' IDENTIFIED BY 'dbpass';
GRANT SELECT, INSERT, UPDATE, DELETE ON Firma.* TO dbuser@localhost;
```

- Palvelinasennuksen yhteydessä määritettiin root-käyttäjätilin salasana.
- Tuolla tilillä on kaikki oikeudet kaikkiin tietokantoihin.
- Tietokannan käyttöä varten kannattaa turvallisuussyistä tehdä erillinen rajoitettu käyttäjätunnus.
- Käyttäjätunnukseen liitetään tarvittavat oikeudet.
- MariaDB sallii oikeuksien määrittämisen tietokannan, taulun ja kentän tasolla. Lisäksi voidaan määrittää globaaleja oikeuksia (esim. oikeus lukea tiedostosta).
- Yllä olevat lauseet luovat dbuser-nimisen käyttäjätunnuksen, jolla on tietueiden haku-, lisäys-, päivitys- ja poisto-oikeudet Firma-tietokantaan.
  - Käyttäjä dbuser tunnistautuu salasanalla dbpass.

Datan muokkaus

- Tietokannan käyttö koostuu:
  - Datan lukemisesta (SELECT)
  - Datan muokkaamisesta (UPDATE, INSERT, DELETE).
- Tietokanta on syytä suunnitella siten, että sen käyttö ei missään tilanteessa edellytä muutoksia rakenteeseen.

Datan päivitys: UPDATE

- UPDATE-sanan perässä määrätään taulu, johon päivitys kohdistuu.
- SET-sanan perässä annetaan kentälle tai kentille uudet arvot.
- WHERE-osassa määritetään ehto, jonka toteutuessa tietuetta päivitetään.

```
1 UPDATE TYONTEKIJA
2 SET Palkka = Palkka + 30
3 WHERE Osasto =
4 (SELECT Numero FROM OSASTO WHERE Nimi='Laskutus');
```

Affected rows: 487

Datan poisto: DELETE

- Poistolause kohdistuu aina kokonaisiin tietueisiin.
- WHERE-sanan edessä määrätään ehto, jonka täyttyessä tietue poistetaan.

```
DELETE
FROM TYONTEKIJA
WHERE Palkka > 8000
AND Numero IN (
   SELECT Haltija
   FROM AUTO
   WHERE Merkki = 'Honda'
)
```

```
Affected rows: 0
```

Skeeman suunnittelusta

- Skeeman eli relaatiokaavion suunnittelu perustuu
  - Yksilötyyppien tunnistamiseen
  - Niiden lukumäärärajoitteiden analysointiin (1:M, M:N, 1:1)
- Se voidaan tehdä
  a. Kohdealueen käsitemaailmaan perustuen yksilö-yhteyskaavioiden eli ER-kaavioiden avulla
  b. Suoraan datan tallennusrakennetta (taulukoita) mallintamalla
  c. Ohjelmointikielen rakenteista (luokat) johtamalla silloin kun tietokanta palvelee yksinomaan tiettyä ohjelmistoa.
     - ORM-kirjasto (esim. Hibernate) voi automatisoida tämän.